

Python编程入门到精通

AI 编程实验室 · 精选教程

¥19.9

生成日期: 2026年05月06日

21 篇精选文章

目录

1. 目录

2. 1. 理解异步编程的核心：协程与事件循环

3. 2. 实际场景：异步HTTP请求

4. 3. 控制并发：限制同时进行的任务数

5. 4. 超💎💎💎处理与错误恢复

6. 5. 异步与同步代码的桥接

7. 总结

8. 一、迭代器：数据流的生产者

9. 二、生成💎💎💎：迭代器的优雅实现

10. 三、实战：处理大文件

11. 四、生成器的进阶用法

12. 五、性能对比与选择建议

13. 总结

14. 一、理解装饰器的核心概念

15. 二、处理带参数的函数

16. 三、保留原函数的元信息

17. 四、带参数的装饰器

18. 五、实际应用场景

19. 六、多个装饰器的组合

20. 总结

Python 编程入门到精通

从零开始的 Python 学习指南，涵盖基础语法、实战项目、最佳实践。

目录

- 1. Python异步编程：asyncio实战 (2026-05-06)
- 2. Python生成器与迭代器：高效处理大数据 (2026-05-06)
- 3. Python装饰器从入门到进阶 (2026-05-06)

声明： 本内容由 AI 编程实验室自动整理生成，仅供参考学习。

=====

第 1 章：Python异步编程：asyncio实战

Python异步编程：asyncio实战

在开发网络爬虫、Web服务或需要处理大量I/O操作的应用时，你是否遇到过程序卡在等待响应上，导致整体性能低下？Python的 `asyncio` 库正是解决这类问题的利器。本文将带你从零掌握 `asyncio` 的核心用法，并通过实战案例提升代码执行效率。

1. 理解异步编程的核心：协程与事件循环

异步编程的核心是**协程**（Coroutine）和**事件循环**（Event Loop）。协程是可以暂停执行并让出控制权的函数，事件循环则负责调度这些协程。

基础示例：定义和运行协程

```
import asyncio

async def greet(name):
    print(f"Hello, {name}!")
    await asyncio.sleep(1) # 模拟I/O等待
    print(f"Goodbye, {name}!")

async def main():
    # 创建协程对象，但不会立即执行
    task1 = asyncio.create_task(greet("Alice"))
    task2 = asyncio.create_task(greet("Bob"))

    # 等待所有任务完成
    await asyncio.gather(task1, task2)

# 运行事件循环
asyncio.run(main())
```

关键点： - `async def` 定义协程函数 - `await` 挂起当前协程，等待另一个协程完成 - `asyncio.run()` 是Python 3.7+推荐的启动入口

运行上述代码，你会看到两个问候语几乎同时输出，而不是顺序执行——这就是异步的魅力。

2. 实际场景：异步HTTP请求

网络请求是最常见的I/O密集型操作。我们使用 `aiohttp` 库来实现异步HTTP请求。

安装依赖

```
pip install aiohttp
```

实战：批量抓取网页标题

```
import asyncio
import aiohttp
from bs4 import BeautifulSoup

async def fetch_title(session, url):
    try:
        async with session.get(url) as response:
            html = await response.text()
            soup = BeautifulSoup(html, 'html.parser')
            title = soup.title.string if soup.title else "No title"
            print(f"{url}: {title[:50]}")
            return title
    except Exception as e:
        print(f"Error fetching {url}: {e}")
        return None

async def main():
    urls = [
        "https://www.python.org",
        "https://www.github.com",
        "https://stackoverflow.com",
        "https://www.reddit.com"
    ]

    async with aiohttp.ClientSession() as session:
        # 创建所有抓取任务
        tasks = [fetch_title(session, url) for url in urls]
        # 并发执行
        results = await asyncio.gather(*tasks)

    print(f"成功抓取 {sum(1 for r in results if r)}/{len(urls)} 个页面")

asyncio.run(main())
```

性能对比：如果使用同步方式，4个请求需要至少4秒（假设每个请求1秒）；而异步方式只需约1秒，效率提升明显。

3. 控制并发：限制同时进行的任务数

直接使用 `asyncio.gather` 会一次性发起所有任务，可能导致目标服务器过载或被封IP。我们需要限制并发数。

使用信号量控制并发

```
import asyncio
import aiohttp

async def fetch_url(semaphore, session, url):
    async with semaphore: # 获取信号量
        async with session.get(url) as response:
            return await response.text()

async def main():
    urls = [f"https://httpbin.org/delay/{i}" for i in range(1, 11)] # 模拟10个慢请求

    # 限制最多3个并发
    semaphore = asyncio.Semaphore(3)

    async with aiohttp.ClientSession() as session:
        tasks = [fetch_url(semaphore, session, url) for url in urls]
        results = await asyncio.gather(*tasks)

    print(f"完成 {len(results)} 个请求")

asyncio.run(main())
```

信号量工作原理：同时最多允许3个协程进入 `async with semaphore` 块，其他协程会排队等待。

4. 超⚡️⚡️⚡️处理与错误恢复

在实际应用中，网络请求可能超时或失败。我们需要优雅地处理这些异常。

带超时的任务管理

```
import asyncio
import aiohttp

async def fetch_with_timeout(session, url, timeout=5):
    try:
        async with asyncio.timeout(timeout): # Python 3.11+ 语法
            async with session.get(url) as response:
                return await response.text()
    except asyncio.TimeoutError:
        print(f"{url} 请求超时")
```

```

        return None
    except Exception as e:
        print(f"{url} 请求失败: {e}")
        return None

# 对于Python 3.10及以下版本, 使用asyncio.wait_for
async def fetch_with_timeout_legacy(session, url, timeout=5):
    try:
        async with session.get(url) as response:
            # 限制读取操作的时间
            return await asyncio.wait_for(response.text(), timeout=timeout)
    except asyncio.TimeoutError:
        print(f"{url} 请求超时")
        return None

```

实战建议：为每个任务设置合理的超时时间，避免单个慢请求拖垮整个程序。

5. 异步与同步代码的桥接

有时我们需要在异步代码中调用同步的第三方库。使用 `loop.run_in_executor` 可以将同步任务交给线程池执行。

混合使用示例

```

import asyncio
import time

def sync_blocking_task(n):
    """模拟一个耗时的CPU操作"""
    time.sleep(n) # 注意: 这里使用时间.sleep模拟阻塞
    return f"完成耗时{n}秒的任务"

async def main():
    loop = asyncio.get_running_loop()

    # 将同步任务提交到默认线程池
    result = await loop.run_in_executor(None, sync_blocking_task, 2)
    print(result)

    # 也可以指定线程池
    from concurrent.futures import ThreadPoolExecutor
    with ThreadPoolExecutor(max_workers=4) as pool:
        results = await asyncio.gather(*[
            loop.run_in_executor(pool, sync_blocking_task, i)
            for i in range(1, 4)
        ])

```



```
print(results)

asyncio.run(main())
```

注意： `run_in_executor` 适用于CPU密集型或阻塞的同步代码，但不要滥用——异步的核心优势是处理I/O等待。

总结

本文带你走过了asyncio实战的四个关键环节：1. **协程与事件循环**：理解 `async/await` 语法和 `asyncio.run` 入口 2. **异步HTTP请求**：使用 `aiohttp` 实现并发网络请求 3. **并发控制**：用 `Semaphore` 限制同时执行的任务数 4. **错误处理**：添加超时和异常处理机制

记住，异步编程不是银弹——它最适合I/O密集型任务（网络请求、文件读写、数据库操作），对于CPU密集型计算（图像处理、加密解密），请考虑使用多进程或多线程。现在，打开你的编辑器，将学到的技术应用到下一个项目中吧！

=====

第 2 章：Python生成器与迭代器：高效处理大数据

Python生成器与迭代器：高效处理大数据

当你的Python程序需要处理数百万行日志、超大CSV文件或无限数据流时，直接使用列表存储所有数据往往会耗尽内存，甚至导致程序崩溃。生成器与迭代器正是解决这类问题的利器——它们通过**惰性计算**方式，只在需要时生成数据，从而大幅降低内存占用。

一、迭代器：数据流的生产者

任何实现了 `__iter__()` 和 `__next__()` 方法的对象都是迭代器。它像一个“数据管道”，每次调用 `next()` 才吐出下一个元素。

基础示例：手动实现迭代器

```
class Counter:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current >= self.end:
            raise StopIteration
        value = self.current
        self.current += 1
        return value

# 使用迭代器
counter = Counter(1, 5)
for num in counter:
    print(num) # 输出: 1 2 3 4
```

关键点： `for` 循环本质上是不断调用 `next()` 直到捕获 `StopIteration` 异常。

二、生成器🔗🔗🔗：迭代器的优雅实现

生成器是Python提供了一种更简洁的迭代器创建方式。用 `yield` 关键字替代 `return`，函数就变成了生成器函数。

对比：列表 vs 生成器

```
# 列表方式（一次性生成所有数据）
def get_squares_list(n):
    return [x**2 for x in range(n)]

# 生成器方式（按需生成）
def get_squares_gen(n):
    for x in range(n):
        yield x**2

# 测试内存占用
import sys
squares_list = get_squares_list(100000)
squares_gen = get_squares_gen(100000)

print(f"列表大小: {sys.getsizeof(squares_list)} bytes") # 约824456
print(f"生成器大小: {sys.getsizeof(squares_gen)} bytes") # 约112
```

实际效果：生成器对象无论生成多少数据，自身大小恒定，而列表会随着数据量线性增长。

生成器表达式：一行代码搞定

```
# 列表推导式
list_comp = [x*2 for x in range(1000)]

# 生成器表达式（只需将方括号改为圆括号）
gen_expr = (x*2 for x in range(1000))

print(type(gen_expr)) # <class 'generator'>
```

三、实战：处理大文件

假设有一个10GB的日志文件 `access.log`，我们需要统计每个IP的访问次数。

错误做法（内存爆炸）

```
def parse_log_bad(filepath):
    with open(filepath) as f:
        lines = f.readlines() # 一次性加载全部内容到内存
        ip_counts = {}
        for line in lines:
            ip = line.split()[0]
```

```
ip_counts[ip] = ip_counts.get(ip, 0) + 1
return ip_counts
```

正确做法（流式处理）

```
def parse_log_good(filepath):
    ip_counts = {}
    with open(filepath) as f:
        for line in f: # 文件对象本身就是迭代器，逐行读取
            ip = line.split()[0]
            ip_counts[ip] = ip_counts.get(ip, 0) + 1
    return ip_counts

# 或者使用生成器函数封装读取逻辑
def read_lines(filepath):
    with open(filepath) as f:
        for line in f:
            yield line.strip()

# 使用生成器处理
for line in read_lines("huge_log.txt"):
    process(line) # 逐行处理，内存占用极低
```

原理：`open()` 返回的文件对象是一个迭代器，`for line in f` 不会将整个文件读入内存，而是每次读取一行。

四、生成器的进阶用法

1. 无限序列生成

```
def fibonacci():
    a, b = 0, 1
    while True: # 无限循环
        yield a
        a, b = b, a + b

# 只取前10个
fib = fibonacci()
first_10 = [next(fib) for _ in range(10)]
print(first_10) # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

2. 生成器管道（数据流水线）

```
# 构建处理管道
def read_data(filepath):
    with open(filepath) as f:
        for line in f:
            yield line.strip()

def filter_empty(data):
    for item in data:
        if item: # 过滤空行
            yield item

def to_uppercase(data):
    for item in data:
        yield item.upper()

# 串联管道
pipeline = to_uppercase(filter_empty(read_data("data.txt")))
for processed_line in pipeline:
    print(processed_line)
```

3. 使用 yield from 简化嵌套生成器

```
def chain(*iterables):
    for it in iterables:
        yield from it # 委托给子迭代器

# 合并多个列表
combined = chain([1,2,3], "abc", [4,5])
print(list(combined)) # [1, 2, 3, 'a', 'b', 'c', 4, 5]
```

五、性能对比与选择建议

场景	推荐方式	原因
数据量 < 1000条	列表	简单直观，访问速度快
数据量 > 10000条	生成器/迭代器	内存占用低，避免OOM
需要多次遍历	列表/重新创建生成器	生成器只能遍历一次
无限数据流	生成器	无法用列表表示无限序列

测试代码：比较列表和生成器的性能

```
import time

# 列表方式
start = time.time()
result = sum([x for x in range(10_000_000)])
print(f"列表耗时: {time.time() - start:.2f}s")

# 生成器方式
start = time.time()
result = sum(x for x in range(10_000_000))
print(f"生成器耗时: {time.time() - start:.2f}s")
# 两者耗时相近，但生成器内存占用几乎为0
```

总结

- **迭代器**是实现了 `__iter__` 和 `__next__` 的对象，支持惰性计算
- **生成器**是用 `yield` 创建的迭代器，更简洁易用
- 处理大文件、无限序列时，优先使用生成器
- 生成器表达式是列表推导式的内存友好版本
- 通过 `yield from` 可以轻松构建数据管道

掌握生成器和迭代器，你的Python程序就能以极低的内存开销处理海量数据——这正是Python在数据处理领域广受欢迎的重要原因之一。

=====

第 3 章：Python装饰器从入门到进阶

Python装饰器从入门到进阶

装饰器是Python中一个强大而优雅的特性，它允许你在不修改原函数代码的情况下，动态地给函数添加新的功能。本文将带你从基础语法到高级应用，彻底掌握装饰器的使用。

一、理解装饰器的核心概念

装饰器本质上是一个接收函数作为参数并返回新函数的高阶函数。让我们从最简单的例子开始：

```
def my_decorator(func):
    def wrapper():
        print("在函数执行前做一些事情")
        func()
        print("在函数执行后做一些事情")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

输出：

```
在函数执行前做一些事情
Hello!
在函数执行后做一些事情
```

`@my_decorator` 语法糖等价于 `say_hello = my_decorator(say_hello)`。 `wrapper` 函数包裹了原函数，在调用前后添加了额外逻辑。

二、处理带参数的函数

上面的装饰器只能处理无参函数。为了让装饰器通用，我们需要使用 `*args` 和 `**kwargs`：

```
def timer_decorator(func):
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
```

```

        result = func(*args, **kwargs)
    end = time.time()
    print(f"{func.__name__} 执行耗时: {end - start:.4f}秒")
    return result
return wrapper

@timer_decorator
def calculate_sum(n):
    return sum(range(n))

@timer_decorator
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

result = calculate_sum(1000000)
print(f"计算结果: {result}")
greet("Alice", "Hi")

```

这个装饰器可以测量任意函数的执行时间，并保持原函数的返回值。

三、保留原函数的元信息

使用 `functools.wraps` 可以保留被装饰函数的名称、文档字符串等元信息：

```

from functools import wraps

def log_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"调用函数: {func.__name__}")
        print(f"参数: {args}, {kwargs}")
        result = func(*args, **kwargs)
        print(f"返回值: {result}")
        return result
    return wrapper

@log_decorator
def add(a, b):
    """计算两个数的和"""
    return a + b

print(f"函数名: {add.__name__}")
print(f"文档: {add.__doc__}")
add(3, 5)

```

没有 `@wraps` 时，`add.__name__` 会变成 `wrapper`，这会导致调试困难。

四、带参数的装饰器

有时我们需要给装饰器传递参数，比如设置日志级别。这需要再嵌套一层函数：

```
def repeat(times):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(times=3)
def say_hi():
    print("Hi!")

say_hi()
```

这里的 `repeat(times=3)` 先用返回 `decorator` 函数，然后 `@decorator` 再应用于 `say_hi`。

五、实际应用场景

1. 权限验证

```
def require_auth(role):
    def decorator(func):
        @wraps(func)
        def wrapper(user, *args, **kwargs):
            if user.get("role") != role:
                raise PermissionError(f"需要{role}权限")
            return func(user, *args, **kwargs)
        return wrapper
    return decorator

@require_auth("admin")
def delete_user(user, user_id):
    print(f"用户 {user_id} 已被删除")

# 测试
admin_user = {"name": "Admin", "role": "admin"}
normal_user = {"name": "User", "role": "user"}
```

```
delete_user(admin_user, 123) # 正常执行
# delete_user(normal_user, 123) # 抛出PermissionError
```

2. 缓存计算结果

```
def cache_result(func):
    cache = {}
    @wraps(func)
    def wrapper(*args):
        if args in cache:
            print(f"使用缓存: {args}")
            return cache[args]
        result = func(*args)
        cache[args] = result
        return result
    return wrapper

@cache_result
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(10)) # 第一次计算
print(fibonacci(10)) # 直接从缓存获取
```

3. 重试机制

```
import time

def retry(max_attempts=3, delay=1):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    if attempt == max_attempts - 1:
                        raise
                    print(f"第{attempt + 1}次失败, {delay}秒后重试...")
                    time.sleep(delay)
            return wrapper
        return decorator

    @retry(max_attempts=3, delay=0.5)
    def unstable_network_call():
```

```
import random
if random.random() < 0.7:
    raise ConnectionError("网络异常")
return "成功获取数据"

print(unstable_network_call())
```

六、多个装饰器的组合

当多个装饰器叠加时，执行顺序❖❖❖从下往上（离函数最近的先执行）：

```
@timer_decorator
@log_decorator
def complex_function(x, y):
    return x ** y

complex_function(2, 10)
```

这相当于 `complex_function = timer_decorator(log_decorator(complex_function))`，先执行 `log_decorator`，再执行 `timer_decorator`。

总结

装饰器是Python中实现AOP（面向切面编程）的优雅方式。核心要点：

1. **基本结构**：装饰器是接收函数返回函数的高阶函数
2. **参数传递**：使用 `*args`、`**kwargs` 保持通用性
3. **元信息保留**：始终使用 `@wraps` 装饰wrapper函数
4. **带参数装饰器**：需要三层嵌套函数
5. **实际应用**：日志、权限、缓存、重试等场景非常实用

掌握装饰器后，你会发现代码复用性和可维护性都得到了显著提升。

✳ 关于本产品

- 来源：AI 编程实验室
- 文章数：3 篇

- 生成时间: 2026-05-06T05:47:06.483Z

如果觉得有帮助，欢迎分享给需要的朋友！

— END —

由 AI 编程实验室自动生成

<https://hugo-xu844.github.io/autopilot-content/>